Desenvolvimento de Sistemas Web II

O contrato gRPC, a API chamando o Go, containers e implantação

DIM0547 — Turma 01 · Sprint 2 · aula em vídeo

Prof. Fernando · UFRN · 2026.2

Onde paramos

Passo	Hoje
14	O contrato: Protocol Buffers e Buf
15	O servidor gRPC em Go
16	A API (Ktor e Quarkus) chama o serviço Go, com prazo, e continua funcionando sem ele
17	Tudo em containers: Dockerfiles, health checks e `docker compose up` do zero
18	Implantação: configuração por ambiente, migração por comando, imagens no GHCR, Render e Neon

Branch `aula`: `./passo.sh 13`.

Passo 14

O contrato: Protocol Buffers e Buf

O contrato `.proto`

- Um arquivo, três linguagens: o Go, o Kotlin e o Java geram o código a partir dele
- Números de campo nunca mudam e nunca são reaproveitados: é o que mantém a compatibilidade
- Mudança incompatível vira `v2`, ao lado da `v1`

📖 **Ref.** Protocol Buffers — proto3

Buf: conferir e gerar

```
```

```
```

- Plugins remotos: nada de `protoc` instalado na máquina
- `buf lint` vai para o CI: a rubrica pede o contrato verde lá
- Os stubs de Kotlin e Java são gerados pelo Gradle e pelo Quarkus, no build

📖 **Ref.** Buf — lint rules

Passo 15

O servidor gRPC: status, prazo, health e reflection

O adaptador: do contrato para o domínio

Situação	Código gRPC	Equivalente HTTP
entrada inválida	`INVALID_ARGUMENT`	400
prazo vencido	`DEADLINE_EXCEEDED`	504
serviço fora do ar	`UNAVAILABLE`	503
erro interno	`INTERNAL`	500

📖 **Ref.** gRPC — Status codes

O prazo atravessa a rede

- Sem a folga, o servidor esperaria até o fim e o cliente veria só `DEADLINE_EXCEEDED`
- O prazo viaja no cabeçalho `grpc-timeout`: dá para vê-lo no log do cliente

📖 **Ref.** gRPC — Deadlines

O servidor e a demonstração

`cmd/avisos/main.go`: registra o serviço, o *health check* padrão e o *reflection*; encerra com `GracefulStop` ao receber SIGTERM.

```
```

Testes do servidor: `bufconn`, uma conexão em memória que passa pela pilha gRPC inteira, sem abrir porta.

gRPC ou REST?

	REST com JSON	gRPC com Protobuf
Contrato	OpenAPI, gerado ou escrito	`.proto`, a fonte de onde o código é gerado
Formato	texto (JSON), legível	binário, compacto
Transporte	HTTP/1.1 ou 2	HTTP/2
Erros	status HTTP e *problem details*	códigos gRPC
Bom para	APIs públicas, navegador, apps	serviço falando com serviço, dentro do sistema

> No projeto, as duas convivem: REST na API que o cliente usa; gRPC entre a API e o serviço Go.

📖 **Ref.** gRPC — Introduction

Passo 16

A API chama o Go

Uma porta nova no domínio

Ktor — `dominio/Tarefa.kt`

Quarkus — `dominio/Avisador.java`

Na rota (ou no recurso), depois de criar:

- O domínio não sabe que existe gRPC: devolve `false` quando não conseguiu, sem exceção
- O teste de arquitetura passa a proibir `io.grpc` e `com.google.protobuf` no domínio

O adaptador no Ktor: stub com corrotinas

```
```

`Aplicacao.kt`: `AvisadorGrpc.para(System.getenv("AVISOS_ENDERECO") ?: "localhost:50051")`, fechado no `ApplicationStopped`.

O adaptador no Quarkus: stub injetado

Stub bloqueante: o recurso já roda numa thread de trabalho.

De onde vêm os stubs

`Ktor` — `build.gradle.kts`

`Quarkus` — `pom.xml`

Um contrato, em `exemplos/protos/`, para os três projetos. Ktor: o BOM `grpc-bom` alinha as versões de `io.grpc`.

Degradação: a API sem o serviço Go

Serviço Go	Resposta da API	No log da API
no ar	`201` em ≈ 0,5 s	`aviso da tarefa 5: [email=true, push=true, chat=true]`
parado	`201` em ≈ 0,02 s	`aviso da tarefa 6 não enviado: UNAVAILABLE`
lento demais	`201` em ≈ 1 s (o prazo)	`não enviado: DEADLINE_EXCEEDED`

- A tarefa é o que importa; o aviso é acessório. Por isso a falha do aviso não vira erro HTTP
- Sem prazo, um serviço lento prenderia cada requisição pelo tempo que ele quisesse

Testes do cliente gRPC

Teste	Ktor	Quarkus
cliente: rápido, lento além do prazo, fora do ar	3	—
rota: criar avisa; aviso que falha não impede a criação	2	2
total da suíte	16	13

Passo 17

Tudo em containers

Dockerfiles em dois estágios

`Go` — 29 MB

`Ktor` — cerca de 600 MB

- O primeiro estágio compila; o segundo só leva o resultado. Ferramentas de build não vão para produção
- O contexto do build da API é `exemplos/`, porque ela precisa de `protos/`

Health checks e o `docker compose`

Serviço	Health check
PostgreSQL	`pg_isready`
API Ktor	`GET /health`
API Quarkus	`GET /q/health` (extensão `quarkus-smallrye-health`, que também verifica o banco)
Go	`avisos saude`: o próprio binário chama o health check gRPC (a imagem não tem shell nem `curl`)

Passo 18

Implantação

A mesma imagem, configurada pelo ambiente

Variável	Desenvolvimento	No compose	Em produção
`DB_URL`	`localhost:5432`	`banco:5432`	a connection string do Neon
`AVISOS_ENDERECO`	`localhost:50051`	`avisos:50051`	o endereço interno do serviço Go
`PORT`	8080	8080	a porta que a plataforma define

- Nada de configuração de ambiente no código: a imagem é a mesma em todo lugar
- Senhas e connection strings são **segredos**: entram no painel da plataforma, nunca no repositório

Migração por comando, antes de a versão nova entrar no ar:

Do commit à imagem publicada

`exemplos/implantacao/ci-e-imagens.yml` (modelo; no projeto, fica em `.github/workflows/`):

```
```

Cada imagem leva a etiqueta do commit: voltar a uma versão é apontar para a etiqueta anterior.

📖 **Ref.** GitHub — Publishing Docker images

No ar: Render e Neon, e um limite importante

`exemplos/implantacao/render.yaml`: a API como web service, o serviço Go como serviço privado, o banco no Neon.

Plano gratuito do Render (consultado em 15/10)	Consequência
serviço privado não tem instância gratuita	o Go num serviço privado custa
web service gratuito não recebe tráfego pela rede privada	o Go como web service gratuito não é alcançável pela API
o endereço público não encaminha gRPC	não dá para chamar o Go pela internet
`preDeployCommand` só em plano pago	no gratuito, a API migra na subida

> Na entrega, o critério **Sistema no ar** pede a API e o banco no ar. O serviço Go roda junto da API no `docker compose`; pô-lo no ar é opcional, registrado numa ADR.

📖 **Ref.** Render — Private network · Render — Free instances

O que a Sprint 2 cobra

Critério	Peso	O que mostrar
Microsserviço Go	25%	responsabilidade justificada (ADR); Go idiomático, `context` e erros
Contrato e gRPC	25%	`.proto` comentado, `buf lint` verde no CI, stubs gerados, chamada com prazo e erro tratado, teste de integração
Sistema no ar	25%	API e banco gerenciado no ar, URL no README, migração por task ou CI, segredos fora do repositório, health check, API que funciona sem o Go (o Go no ar é opcional)
Ambiente e testes	15%	`docker compose up` do zero; unitários, integração e teste de arquitetura, verdes local e no CI
Documentação	10%	README completo (roda em menos de 10 min), OpenAPI por URL, 2 ADRs novas, `LICENSE`

Prova em **11/11**; *daily meetings* em 23 e 25/11; **entrega em 30/11**.

Onde estudar depois

- Leitura da sprint: `leituras/web2-s2.md`, capítulos 6 a 9
- Passos: `exemplos/avisos-go/PASSOS.md` · branch `aula`, tags `passo-16` a `passo-18`
- gRPC — Kotlin quick start · Quarkus — gRPC clients
- Docker — multi-stage builds · Compose — services
- Render — Blueprint YAML · Neon — Connect from any application